GEORG-AUGUST-UNIVERSITÄT
GÖTTINGEN IN PUBLICA COMMODA SEIT 1737

Top-down Syntax Analysis

Roland Leißa
(based on slides by S. Hack, R. Wilhelm, and M. Sagiv)

University of Göttingen

Top-Down Syntax Analysis

Input: A sequence of symbols (tokens)

Output: A syntax tree or an error message

- Read input from left to right
- Construct the syntax tree in a top-down manner starting with a node labeled with the start symbol
- **until** input accepted (or error) **do**
  - Predict expansion for the actual leftmost nonterminal (maybe using some lookahead into the remaining input) or
  - Verify predicted terminal symbol against next symbol of the remaining input
- Finds leftmost derivations

Eliminating Left Recursion

Consider the following regular expression:

$$A_r = \beta\alpha^*$$

As left-recursive grammar:

$$A \to A\alpha \mid \beta$$

Or equivalently as right-recursive grammar:

$$A \to \beta A'$$
$$A' \to \varepsilon \mid \alpha A'$$

Example

$$\underbrace{E}_{A} \to \underbrace{E}_{A}\underbrace{+T}_{\alpha} \mid \underbrace{T}_{\beta}$$

Using this rule:

$$\underbrace{E}_{A} \to \underbrace{T}_{\beta}\underbrace{E'}_{A'}$$
$$\underbrace{E'}_{A'} \to \varepsilon \mid \underbrace{+T}_{\alpha}\underbrace{E'}_{A'}$$

Grammar for Arithmetic Expressions

Left factored grammar G_2, i.e. left recursion removed.

$S \to E$	
$E \to TE'$	E generates T with a continuation E'
$E' \to +E \mid \varepsilon$	E' generates possibly empty sequence of $+T$s
$T \to FT'$	T generates F with a continuation T'
$T' \to *T \mid \varepsilon$	T' generates possibly empty sequence of $*F$s
$F \to \mathsf{id} \mid (E)$	

G_2 defines the same language as G_0 und G_1.

But the parse tree is not so suitable as an abstract syntax tree!

Recursive Descent Parsing

- parser is a program
- a procedure `parseX` for each non-terminal *X*
  - parses words for non-terminal *X*,
  - starts with the first symbol read (into variable *nextsym*)
  - ends with the following symbol read (into variable *nextsym*)
- uses one symbol lookahead into the remaining input.
- uses the *FiFo* sets to make the expansion transitions deterministic

$$FiFo(N \to \alpha) = First_1(\alpha) \oplus_1 Follow_1(N)$$

(Note: $a \oplus_k b$ are the first k chars of ab)

The $First_1$ Sets

- A production $N \to \alpha$ is applicable for symbols that "begin" with α
- Example: Arithmetic Expressions, Grammar G_2
  - The production $F \to id$ is applied when the current symbol is **id**
  - The production $F \to (E)$ is applied when the current symbol is **(**
  - The production $T \to F$ is applied when the current symbol is **id** or **(**
- Formal definition:

$$First_1(\alpha) = \{1 : w \mid \alpha \stackrel{*}{\Longrightarrow} w, w \in V_T^*\}$$

The $Follow_1$ Sets

- A production $N \to \varepsilon$ is applicable for symbols that "can follow" N in some derivation
- Example: Arithmetic Expressions, Grammar G_2
  - The production $E' \to \varepsilon$ is applied for symbols **#** and **)**
  - The production $T' \to \varepsilon$ is applied for symbols **#**, **)** and **+**
- Formal definition:

$$Follow_1(N) = \{a \in V_T \mid \exists \alpha, \gamma : S \overset{*}{\Longrightarrow} \alpha N a \gamma\}$$

Definitions

Let $k \geq 1$

- k-prefix of a word $w = a_1 \dots a_n$

$$k : w = \begin{cases} a_1 \dots a_n & \text{if } n \leq k \\ a_1 \dots a_k & \text{otherwise} \end{cases}$$

- k-concatenation

$$\oplus_k : V^* \times V^* \to V^{\leq k}, \text{ defined by } u \oplus_k v = k : uv$$

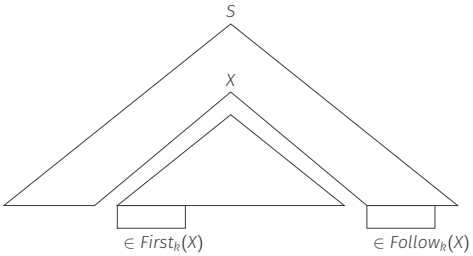
- extended to languages

$$k : L = \{k : w \mid w \in L\}$$

$$L_1 \oplus_k L_2 = \{x \oplus_k y \mid x \in L_1, y \in L_2\}$$

$$V^{\leq k} = \bigcup_{i=1}^{k} V^i \text{ set of words of length at most } k$$

$First_k$ and $Follow_k$



- set of k–prefixes of terminal words for α

$$First_k : (V_N \cup V_T)^* \to 2^{V_T^{\leq k}} \qquad First_k(\alpha) = \{k : u \mid \alpha \overset{*}{\Longrightarrow} u\}$$

- set of k–prefixes of terminal words that may immediately follow X

$$Follow_k : V_N \to 2^{V_{T\#}^{\leq k}}$$

$$Follow_k(X) = \{w \mid S \overset{*}{\Longrightarrow} \beta X \gamma \text{ and } w \in First_k(\gamma)\}$$

Parser for G_2

```
program parser;
var  nextsym: string;
proc  scan;
 {reads next input symbol into  nextsym}
proc  error (message: string);
 {issues error message and stops parser}
proc  accept; {terminates successfully}

proc  parseS;
 begin
  parseE
 end ;

proc  parseE;
 begin
  parseT; parseE'
 end ;
```

			$First_1$	$Follow_1$
S	$\rightarrow$	E	$\{\mathsf{id}, (\}$	$\{\#\}$
E	$\rightarrow$	TE'	$\{\mathsf{id}, (\}$	$\{\#,)\}$
E'	$\rightarrow$	$+E \mid \varepsilon$	$\{+, \varepsilon\}$	$\{\#,)\}$
T	$\rightarrow$	FT'	$\{\mathsf{id}, (\}$	$\{\#,), +\}$
T'	$\rightarrow$	$*T \mid \varepsilon$	$\{*, \varepsilon\}$	$\{\#,), +\}$
F	$\rightarrow$	$\mathsf{id} \mid (E)$	$\{\mathsf{id}, (\}$	$\{\#,), *, +\}$

```
proc parseE';
 begin
   case nextsym in
    {"+"}:if nextsym = "+ "
            then scan
            else error( "+ expected") fi ; parseE;
     otherwise ;
   endcase
 end ;

proc parseT;
 begin parseF; parseT' end ;

proc parseT';
 begin
   case nextsym in
    {"*"}:if nextsym = "*"
            then scan
            else error( "* expected") fi ; parseT;
     otherwise ;
   endcase
 end ;
```

			$First_1$	$Follow_1$
S	$\rightarrow$	E	$\{\mathbf{id}, (\}$	$\{\#\}$
E	$\rightarrow$	TE'	$\{\mathbf{id}, (\}$	$\{\#,)\}$
E'	$\rightarrow$	$+E \mid \varepsilon$	$\{+, \varepsilon\}$	$\{\#,)\}$
T	$\rightarrow$	FT'	$\{\mathbf{id}, (\}$	$\{\#,), +\}$
T'	$\rightarrow$	$*T \mid \varepsilon$	$\{*, \varepsilon\}$	$\{\#,), +\}$
F	$\rightarrow$	$\mathbf{id} \mid (E)$	$\{\mathbf{id}, (\}$	$\{\#,), *, +\}$

```
proc parseF;
 begin
   case nextsym in
    {"("}: if nextsym = "("
           then scan
           else error( "( expected") fi ; parseE;
           if nextsym = ")"
           then scan else error(" ) expected") fi;
    otherwise if nextsym ="id"
           then scan else error("id expected") fi;
   endcase
 end ;

begin
scan; parseS;
if nextsym = "#" then accept else error("# expected") fi
end .
```

			$First_1$	$Follow_1$
S	$\rightarrow$	E	$\{\mathsf{id}, (\}$	$\{\#\}$
E	$\rightarrow$	TE'	$\{\mathsf{id}, (\}$	$\{\#,)\}$
E'	$\rightarrow$	$+E \mid \varepsilon$	$\{+, \varepsilon\}$	$\{\#,)\}$
T	$\rightarrow$	FT'	$\{\mathsf{id}, (\}$	$\{\#,), +\}$
T'	$\rightarrow$	$*T \mid \varepsilon$	$\{*, \varepsilon\}$	$\{\#,), +\}$
F	$\rightarrow$	$\mathsf{id} \mid (E)$	$\{\mathsf{id}, (\}$	$\{\#,), *, +\}$

How to Construct such a Parser Program

- Code was automatically generated from the grammar and the *FiFo* sets.
- The program generating the parser has the functions:

N_prog	:	$V_N \to$ code	nonterminals
C_prog	:	$(V_N \cup V_T)^* \to$ code	concatenations
S_prog	:	$V_N \cup V_T \to$ code	symbols

Parser Schema

```
program parser;
    var nextsym: symbol;
    proc scan;
        (* reads next input symbol into nextsym *)
    proc error(message: string);
        (* issues error message and stops the parser *)
    proc accept;
        (* terminates parser successfully *)

    N_prog(X0);                                   (* X0 start symbol *)
    N_prog(X1);
        ⋮
    N_prog(Xn);
```

```
begin
    scan;
    parse X_0;
    if nextsym = "#"
        then accept
        else error("...")
    fi
end
```

The Non-terminal Procedures

N = Non-terminal, C = Concatenation, S = Symbol

N_prog(X) = $\qquad$ (* $X \to \alpha_1|\alpha_2|\cdots|\alpha_{k-1}|\alpha_k$ *)

proc parse$X$;
begin
case nextsym **in**
FiFo($X \to \alpha_1$) : C_progr(α_1);
FiFo($X \to \alpha_2$) : C_progr(α_2);
$\vdots$
FiFo($X \to \alpha_{k-1}$) : C_progr(α_{k-1});
otherwise C_progr(α_k);
endcase
end ;

C_progr($\alpha_1\alpha_2\cdots\alpha_k$) =

S_progr(α_1); S_progr(α_2); ... S_progr(α_k);

S_progr(a) =

if nextsym = a **then** scan

else error ("a expected")

fi

S_progr(Y) = parse$Y$

FiFo–sets have to be disjoint (LL(1)–grammar)

A Generative Solution

Generate the control of a deterministic PDA from the grammar and the *FiFo* sets.

- At compiler–generation time construct a table M
  $M: V_N \times V_T \to P$
  $M[N, a]$ is the production used to expand nonterminal N when the current symbol is a
- For some grammars report that the table cannot be constructed. The compiler writer can then decide to:
  - change the grammar (but not the language)
  - use a more general parser-generator
  - “Patch” the table (manually or using some rules)

Creating the table

Input: cfg G, $First_1$ und $Follow_1$ for G.

Output: The parsing table M or an indication that such a table cannot be constructed

M is constructed as follows:

- For all $X \to \alpha \in P$ and $a \in First_1(\alpha)$, set $M[X, a] = (X \to \alpha)$
- If $\varepsilon \in First_1(\alpha)$, for all $b \in Follow_1(X)$, set $M[X, b] = (X \to \alpha)$
- Set all other entries of M to *error*

Parser table cannot be constructed if at least one entry is set twice.
Then, G is not LL(1)

Example – Arithmetic Expressions

	id	$+$	$*$	$($	$)$	$\#$
S	$S \to E$			$S \to E$		
E	$E \to TE'$			$E \to TE'$		
E'		$E' \to +E$			$E' \to \varepsilon$	$E' \to \varepsilon$
T	$T \to FT'$			$T \to FT'$		
T'		$T' \to \varepsilon$	$T' \to *T$		$T' \to \varepsilon$	$T' \to \varepsilon$
F	$F \to \mathsf{id}$			$F \to (E)$		

Empty cells indicate error!

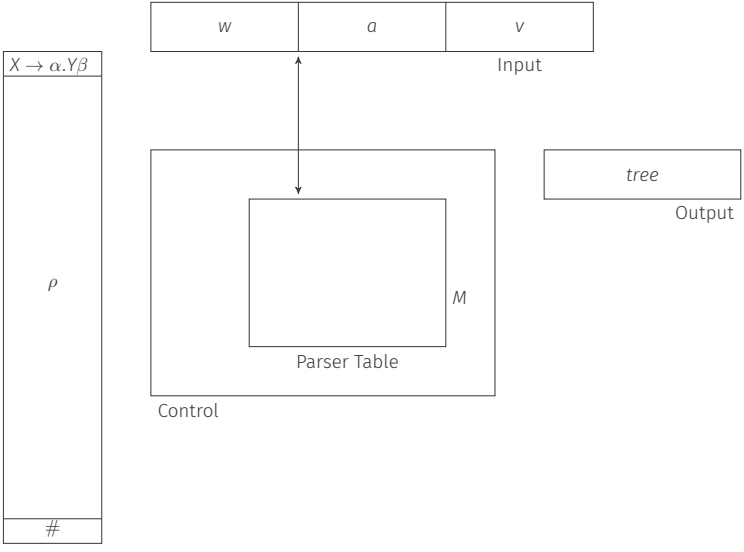
			$First_1$	$Follow_1$
S	$\to$	E	$\{\mathsf{id}, (\}$	$\{\#\}$
E	$\to$	TE'	$\{\mathsf{id}, (\}$	$\{\#,)\}$
E'	$\to$	$+E \mid \varepsilon$	$\{+, \varepsilon\}$	$\{\#,)\}$
T	$\to$	FT'	$\{\mathsf{id}, (\}$	$\{\#,), +\}$
T'	$\to$	$*T \mid \varepsilon$	$\{*, \varepsilon\}$	$\{\#,), +\}$
F	$\to$	$\mathsf{id} \mid (E)$	$\{\mathsf{id}, (\}$	$\{\#,), *, +\}$

LL-Parser Driver (interprets the table M)

```
program parser;
    var nextsym: symbol;
    var st: stack of item;
    proc scan;
        (* reads next input symbol into nextsym *)
    proc error (message: string);
        (* issues error message and stops the parser *)
    proc accept;
        (* terminates parser successfully *)
    proc reduce;
        (* replaces [X → β.Yγ][Y → α.] by [X → βY.γ] *)
    proc pop;
        (* removes topmost item from st *)
    proc push ( i : item);
        (* pushes i onto st *)
    proc replaceby ( i: item);
        (* replaces topmost item of st by i *)
```

```
begin
    scan; push( [S' → .S] );
    while nextsym ≠ "#" do
    case top in
    [X → β.aγ]:   if nextsym = a
                  then scan; replaceby([X → βa.γ])
                  else error fi ;
    [X → β.Yγ] :  if M[Y, nextsym] = (Y → α)
                  then push([Y → .α])
                  else error fi ;
    [X → α.]:     reduce;
    [S' → S.] :   if nextsym = "#" then accept
                  else error fi
    endcase
    od
end .
```

Explicit Stack Deterministic Pushdown Automaton



Disclaimer

What you have just seen is only in parts applicable/encouraged for the project:

- You don't want to use an explicit PDA.
- Writing a parser program by hand for your subset of C is most likely faster and easier to debug than writing a parser generator.
- You don't want to use top-down parsing for expressions (but for statements!) $\rightsquigarrow$ next lecture
- A look-ahead of 1 is not sufficient for (our subset of) the C grammar.

LL(k) Grammar

Goal: formalizing our intuition when the expand-transitions of the Item-Pushdown-Automaton can be made deterministic.

Means: k-symbol lookahead into the remaining input.

LL(k) Grammar

- Let $G = (V_N, V_T, P, S)$ be a cfg and k be a natural number.
  G is an LL(k) **grammar** iff the following holds:

  if for all pairs of leftmost derivations

$$S \overset{*}{\underset{lm}{\Longrightarrow}} uY\alpha \underset{lm}{\Longrightarrow} u\beta\alpha \overset{*}{\underset{lm}{\Longrightarrow}} ux \quad \text{and}$$
$$S \overset{*}{\underset{lm}{\Longrightarrow}} uY\alpha \underset{lm}{\Longrightarrow} u\gamma\alpha \overset{*}{\underset{lm}{\Longrightarrow}} uy \quad \text{and}$$

  if $k : x = k : y$, then $\beta = \gamma$.

- The expansion of the leftmost non-terminal is always uniquely determined by
  - the consumed part of the input and
  - the next k symbols of the remaining input

Example 1

Let G_1 be the cfg with the productions

$$
\begin{aligned}
\mathit{STAT} \rightarrow\ & \textsf{if}\ \ \textsf{id}\ \textsf{then}\ \mathit{STAT}\ \textsf{else}\ \mathit{STAT}\ \textsf{fi} \mid \\
& \textsf{while}\ \ \textsf{id}\ \textsf{do}\ \mathit{STAT}\ \textsf{od} \mid \\
& \textsf{begin}\ \mathit{STAT}\ \textsf{end} \mid \\
& \textsf{id} := \textsf{id}
\end{aligned}
$$

Example 1

Let G_1 be the cfg with the productions

$$
\begin{array}{lll}
\mathit{STAT} \rightarrow & \textsf{if id then } \mathit{STAT} \textsf{ else } \mathit{STAT} \textsf{ fi} & | \\
& \textsf{while id do } \mathit{STAT} \textsf{ od} & | \\
& \textsf{begin } \mathit{STAT} \textsf{ end} & | \\
& \textsf{id} := \textsf{id} &
\end{array}
$$

G_1 is an LL(1)-grammar

$$
\begin{array}{lllll}
\mathit{STAT} & \overset{*}{\underset{lm}{\Longrightarrow}} & w\,\mathit{STAT}\,\alpha & \underset{lm}{\Longrightarrow} & w\,\beta\,\alpha \quad \overset{*}{\underset{lm}{\Longrightarrow}} \quad w\,x \\
\mathit{STAT} & \overset{*}{\underset{lm}{\Longrightarrow}} & w\,\mathit{STAT}\,\alpha & \underset{lm}{\Longrightarrow} & w\,\gamma\,\alpha \quad \overset{*}{\underset{lm}{\Longrightarrow}} \quad w\,y
\end{array}
$$

From $1 : x = 1 : y$ follows $\beta = \gamma$,
e.g., from $1 : x = 1 : y =$ **if** follows $\beta = \gamma =$ "**if id then** *STAT* **else** *STAT* **fi**"

Example 2

Let G_2 be the cfg with the productions

$$
\begin{array}{lll}
\mathit{STAT} \rightarrow & \textsf{if id then } \mathit{STAT} \textsf{ else } \mathit{STAT} \textsf{ fi} & | \\
& \textsf{while id do } \mathit{STAT} \textsf{ od} & | \\
& \textsf{begin } \mathit{STAT} \textsf{ end} & | \\
& \textsf{id} := \textsf{id} & | \\
& \textsf{id}\colon \mathit{STAT} & | \quad (* \text{ labeled statem. } *) \\
& \textsf{id}\,(\textsf{id}) & \quad (* \text{ procedure call } *)
\end{array}
$$

Example 2 (cont'd)

G_2 is not an LL(1)–grammar.

$$STAT \overset{*}{\underset{lm}{\Longrightarrow}} w\ STAT\ \alpha \underset{lm}{\Longrightarrow} w\ \overbrace{\mathsf{id} := \mathsf{id}}^{\beta}\ \alpha \overset{*}{\underset{lm}{\Longrightarrow}} wx$$

$$STAT \overset{*}{\underset{lm}{\Longrightarrow}} w\ STAT\ \alpha \underset{lm}{\Longrightarrow} w\ \overbrace{\mathsf{id} : STAT}^{\gamma}\ \alpha \overset{*}{\underset{lm}{\Longrightarrow}} wy$$

$$STAT \overset{*}{\underset{lm}{\Longrightarrow}} w\ STAT\ \alpha \underset{lm}{\Longrightarrow} w\ \overbrace{\mathsf{id}(\mathsf{id})}^{\delta}\ \alpha \overset{*}{\underset{lm}{\Longrightarrow}} wz$$

and $1 : x = 1 : y = 1 : z = \text{"}\mathsf{id}\text{"}$,
and β, γ, δ are pairwise different.

G_2 is an LL(2)–grammar.
$2 : x = \text{"}\mathsf{id}\ :=\text{"}$, $2 : y = \text{"}\mathsf{id}\ :\text{"}$, $2 : z = \text{"}\mathsf{id}(\text{"}$ are pairwise different.

Example 3

Let G_3 have the productions

$$
\begin{array}{lll}
\mathit{STAT} & \rightarrow & \textsf{if id then } \mathit{STAT} \textsf{ else } \mathit{STAT} \textsf{ fi} \quad | \\
 & & \textsf{while id do } \mathit{STAT} \textsf{ od} \quad | \\
 & & \textsf{begin } \mathit{STAT} \textsf{ end} \quad | \\
 & & \mathit{VAR} := \mathit{VAR} \quad | \\
 & & \textsf{id}(\mathit{IDLIST}) \qquad (* \text{ procedure call } *) \\
\mathit{VAR} & \rightarrow & \textsf{id} \mid \textsf{id}(\mathit{IDLIST}) \qquad (* \text{ indexed variable } *) \\
\mathit{IDLIST} & \rightarrow & \textsf{id} \mid \textsf{id}, \mathit{IDLIST}
\end{array}
$$

Example 3

Let G_3 have the productions

$$
\begin{array}{lcll}
\mathit{STAT} & \to & \textsf{if id then } \mathit{STAT} \textsf{ else } \mathit{STAT} \textsf{ fi} & | \\
 & & \textsf{while id do } \mathit{STAT} \textsf{ od} & | \\
 & & \textsf{begin } \mathit{STAT} \textsf{ end} & | \\
 & & \mathit{VAR} := \mathit{VAR} & | \\
 & & \textsf{id}(\mathit{IDLIST}) & (* \text{ procedure call } *) \\
\mathit{VAR} & \to & \textsf{id} \mid \textsf{id}(\mathit{IDLIST}) & (* \text{ indexed variable } *) \\
\mathit{IDLIST} & \to & \textsf{id} \mid \textsf{id}, \mathit{IDLIST} &
\end{array}
$$

G_3 is not an LL(k)–grammar for any k.

Proof

Assume G_3 to be LL(k) for a $k > 0$.

Let $STAT \Rightarrow \beta \overset{*}{\underset{lm}{\Longrightarrow}} x$ and $STAT \Rightarrow \gamma \overset{*}{\underset{lm}{\Longrightarrow}} y$ with

$$x = \mathsf{id}\,(\underbrace{\mathsf{id}, \mathsf{id}, \ldots, \mathsf{id}}_{k-2 \text{ times}}) := \mathsf{id} \text{ and } y = \mathsf{id}\,(\underbrace{\mathsf{id}, \mathsf{id}, \ldots, \mathsf{id}}_{k-2 \text{ times}})$$

Then $k : x = k : y$,
but $\beta = \text{"}VAR := VAR\text{"} \neq \gamma = \text{"}\mathsf{id}\,(IDLIST)\text{"}$.

Transforming to LL(k)

Factorization creates an LL(2)–grammar, equivalent to G_3.

The productions

$$\mathit{STAT} \rightarrow \mathit{VAR} := \mathit{VAR} \mid \mathsf{id}(\mathit{IDLIST})$$

are replaced by

$$\begin{aligned} \mathit{STAT} &\rightarrow \mathit{ASSPROC} \mid \mathsf{id} := \mathit{VAR} \\ \mathit{ASSPROC} &\rightarrow \mathsf{id}(\mathit{IDLIST})\ \mathit{APREST} \\ \mathit{APREST} &\rightarrow := \mathit{VAR} \mid \varepsilon \end{aligned}$$

A non-LL(k) Language

$$G_4 = (\{S, A, B\}, \{0, 1, a, b\}, P_4, S)$$

$$P_4 = \left\{ \begin{array}{lcl} S & \rightarrow & A \mid B \\ A & \rightarrow & aAb \mid 0 \\ B & \rightarrow & aBbb \mid 1 \end{array} \right\}$$

$$L(G_4) = \{a^n 0 b^n \mid n \geq 0\} \cup \{a^n 1 b^{2n} \mid n \geq 0\}$$

G_4 is not LL(k) for any k. Consider the two leftmost derivations

$$S \underset{lm}{\overset{0}{\Longrightarrow}} S \underset{lm}{\Longrightarrow} A \underset{lm}{\overset{*}{\Longrightarrow}} a^k 0 b^k$$

$$S \underset{lm}{\overset{0}{\Longrightarrow}} S \underset{lm}{\Longrightarrow} B \underset{lm}{\overset{*}{\Longrightarrow}} a^k 1 b^{2k}$$

With $u = \alpha = \varepsilon,\ \beta = A,\ \gamma = B, x = a^k 0 b^k,\ y = a^k 1 b^{2k}$ it holds $k : x = k : y$, but $\beta \neq \gamma$. Since k can be chosen arbitrarily, we have G_4 is not LL(k) for any k.
There even is no LL(k) grammar for $L(G_4)$ for any k.

LL(k) Conditions

Theorem

G is LL(1) iff for different productions $A \to \beta$ *and* $A \to \gamma$

$First_1(\beta) \oplus_1 Follow_1(A) \cap First_1(\gamma) \oplus_1 Follow_1(A) = \emptyset$

Corollary

G is LL(1) iff for all alternatives $A \to \alpha_1 | \ldots | \alpha_n$*:*

1. $First_1(\alpha_1), \ldots, First_1(\alpha_n)$ *are pairwise disjoint; in particular, at most one of them may contain* ε
2. $\alpha_i \overset{*}{\Longrightarrow} \varepsilon$ *implies:*

   $First_1(\alpha_j) \cap Follow_1(A) = \emptyset$ *for* $1 \leq j \leq n,\ j \neq i$.

The Theorem was used in the parser construction!

Further Definitions and Theorems

- G is called a strong LL(k)-grammar (SLL(k)) if for each two different productions $A \to \beta$ and $A \to \gamma$

$$First_k(\beta) \oplus_k Follow_k(A) \cap First_k(\gamma) \oplus_k Follow_k(A) = \emptyset$$

- SLL(1) = LL(1)
- A production is called directly left recursive
  if it has the form $A \to A\alpha$
- A non-terminal A is called left recursive if it has a derivation $A \overset{+}{\Longrightarrow} A\alpha$.
- A cfg G is called left recursive
  if G contains at least one left recursive non-terminal
- G is not LL(k) for any k if G is left recursive
- G is not ambiguous if G is LL(k)grammar

Links

- $First_1$ and $Follow_1$ computation:
  `https://www.cs.uaf.edu/~cs331/notes/FirstFollow.pdf`
- `https://cyberzhg.github.io/toolbox/`